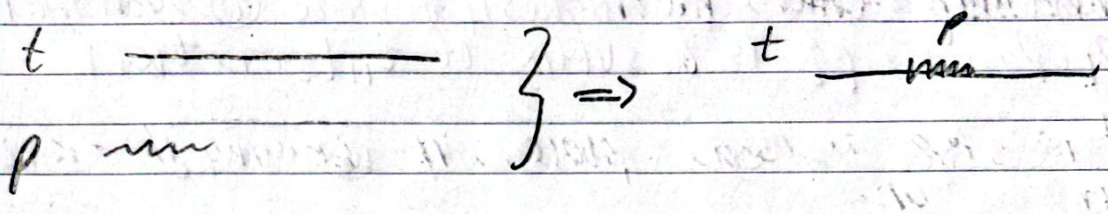
10/29/26

Lecture KMP Algorithm Part II

Problem: Given a pattern $p = p_1, p_2, \ldots, p_L$

a text $t = t_1, t_2, \ldots, t_K$

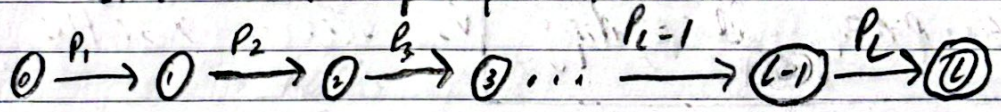
Complete the first exact occurrence of p in t.



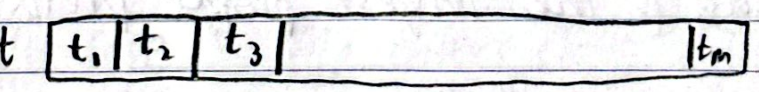
$O(m * L)$

sliding Window

Skeleton DFA M_p : p defines DFA



Input Tape.



Failure Function

Algorithm for computing the failure function f of p.

An Example $p = aa\ bb\ aa\ b$

The failure function f for p

$p = \overset{1}{a}\overset{2}{a}\overset{3}{b}\overset{4}{b}\overset{5}{a}\overset{6}{a}\overset{7}{b}$

j	1	2	3	4	5	6	7
$f(j)$	0	1	0	0	1	2	3

When a mismatch occurs $t_{k+1} \neq p_{j+1}$

then M_p enters the highest # state (i) such that $p_1 p_2 \cdots p_i$ is a suffix of $t_1 t_2 \cdots t_{k+1}$

$p_2 \cdots p_i$

To help determine i the M_p has associated w/ it an integer valued function f that calls the failure function such that $f(j)$ is the largest s less than j for which $p_1 \cdots p_s$ is a suffix of $p_1 \cdots p_j$

$$p_1 p_2 \cdots p_s = p_{j-s+1} \cdots p_{j-1} p_j$$

If there is no such $s \geq 1$ then $f(j) = 0$

$f(7) = 3$ since aab is the longest proper prefix of aabbaab

We will present an algorithm to compute the failure function.

To see how the failure-function is used by M_p let us define $f^{(m)}(j)$ as follows

i) $f^{(i)}(j) = f(j)$

ii) $f^{m}(j) = f(f^{(m-1)}(j))$ for m that is $f^{(m)}(j) = f$

applied m times to j. Ex. $f(6)=2$
$f(f(6)) = f(2)$

M_p = "non-deterministic"

Suppose that M_p is in state (j) having read $t_1, t_2, \ldots, t_k$ on the input tape and $t_{k+1} \neq p_{j+1}$
At this point, M_p applies the failure function f repeatedly to j until it finds the smallest value of m for which either:

Case 1: $f^{(m)}(j) = v$ and $t_{k+1} = p_{v+1}$ OR

Case 2: $f^{(m)}(j) = 0$ and $t_{k+1} \neq p_1$

That is M_p backs up through states $f^{(1)}(j), f^{(2)}(j)$ until → Case 1
↓ Case 2 holds for $f^{(m)}(j)$ but not for $f^{(m-1)}(j)$.

In Case 1: M_p enters state (v+1).
In Case 2: M_p enters state (0).
In either case, the tape head advances to the next letter on the tape: t_{k+2}

In case 1: it is easy to see that if $p_1 p_2 \cdots p_j$ was the largest prefix of p that is a suffix of $t_1, t_2, \ldots, t_k$ then (*)

$p_1 p_2 \ldots p_{f(j)+1}$ is the longest prefix of p that is a suffix of $t_1, t_2, \ldots, t_k, t_{k+1}$.

In case 2 - no prefix of p is a suffix of $t_1, t_2, \ldots, t_k, t_{k+1}$.

Next: MP proceeds processing the symbol t_{k+2} and so on.

MP continues this until it enters the final state (e) in which case p occurs in exactly in t.

or MP continues and reaches the end of t & never enters state (e) - p does NOT occur exactly in t.

The Failure Function Algorithm

Input: $p = p_1 p_2 \ldots p_l$ $l \geq 1$

Output: f_l the failure function of p

Begin

```
f(1)=0, i=0;
For j=2 until l Do:
    Begin
        i=f(j)-1
```

While:

$p_j \neq p_{i+1}$ AND $i > 0$

Do:

$i = f(i)$

If $p_j \neq p_{i+1}$ AND $i = 0$

Then:

$f(j) = 0$

Else:

$f(j) = i + 1$

End (Terminate 1st Loop)

End.

Scanned with CamScanner